Implementaciones de herramientas de aprendizaje autónomo en la clasificación y descubrimiento de posibles aislantes topológicos y topológicos magnéticos

Índice

+ Motivación
+ Generación una data Base
  - Obtención de los datos base
  - Creación y obtención de features
+ Construcción de una red de tipo árbol de decisión
+ Construcción de una red de tipo XGboost
+ Construcción de una red neuronal
  - Red de features simples
  - Red de features especializados

Motivación

La motivación de este trabajo es dado por la necesidad de acotar la búsqueda en el ámbito de los aislantes topológicos, esto debido a lo tedioso del proceso de búsqueda y caracterización usando primero principios (DFT) eso sin mencionar los costos experimentales de realizar una posible aleación la cual no logre obtener la fase deseada.

Siendo que este estudio además se puede expandir a la búsqueda y caracterización de diferentes características en diferentes materiales

Generación una data Base

- Uno de los principales problemas de crear redes neuronales son las bases de datos, que son necesarias para el proceso de entrenamiento, para el caso de aislantes topológicos y topológicos magnéticos este reto se hace una mas grande ya que el número de reportados de los primero no rebasa las centenas y en el caso de los segundos el número no asciende de las decenas.

Obtención de los datos base

- Por esta razón se optó por tomar la base de datos contrastada de topologicalquantumchemistry, este es un trabajo realizado por investigadores de la **Princeton University, Donostia International Physics Center** y otros, esta base de datos que mezcla las aleaciones topológicas comprobadas experimentalmente (las cuales son pocas) con las probadas teóricamente usando Topological Quantum Chemistry, siendo este un modelo donde se buscan obtener representaciones de bandas irreducibles, lo cual confirma la topología, este método es pesada y requiere información completa de las bandas, energías de las aleaciones, simetrías y otros valores específicos, lo cual pese a ser complejo le da validez al mismo.

Si existe la Topological Quantum Chemistry, que aporta el tramiento con redes?

- En la base de datos de ellos tiene hasta el día de esta charla **38184** materiales probado, pero solo **6109** han sido topológicos aproximadamen un **15%** de acierto, lo que esta propuesta puede lograr es limitar la búsqueda usando redes neuronales, no como forma de reemplazar el método de ellos si no minimizar el fallo y la perdida de tiempo

Para este punto espero ya haberlo convencido que no he estado perdiendo el tiempo y que lo que hago puede ser al menos mínimamente util, ahora... como aplico esto?

Creación y obtención de features

- Definimos un features como las características o valores que ingresamos a la red, de la base datos ya mencionada solo tomamos 2 features
    1) Nombre de aleación
    2) Es topológico?
- Esto para todo los topológicos de la base de datos adicionalmente seleccionamos aleatoriamente la misma cantidad pero de materiales triviales.

Features de simetrías espaciales

- Como se sabe teóricamente, la característica que resguarda la topología de las aleaciones son las simetrías espaciales. Por ende, para caracterizar la topología espacial de las aleaciones se extrajo 3 tipos de features. Los cuales son:

    3) Grupo espacial

    4) Tipo de red (acá metimos los 7 tipos posibles y una categoría extra para desconocido, como variables one-hot)

    5) Es centrosimetrica? (o simetría de grupo de inversión)

- Estos datos se extrajeron usando una API KEY de materials Project

Features de inversión de banda

- Para intentar caracterizar la topología de cada aleación se partió con considerar la característica de el acople spin-orbita (SOC) para esto, como sabemos típicamente este valor va de la forma

$$H_{SOC} \propto Z^4$$

- Por esta razón extraemos 4 features de nuestra aleación

  6) Z_{prome}
  7) Z_{diff}
  8) Z_{sum}
  9) Z_{max}^4

- Estos datos fueron extraídos usando la librería matminer y pymatgen de python

Features químicos de distancia interatómica

- Otra valos importante que medio el tipo de enlance de las aleaciones y por ende su estructura de bandas es la distancia interatómica, para tener un aproximado de este valore extraemos el radio atómico :

    10) Radio atómico promedio

    11) Rango de los radios

- Estos datos fueron extraídos usando la librería matminer y pymatgen de python

Features químicos de inversión de bandas

- Como en los aislantes topológicos la inversión de bandas de valencia y conducción es clave, se hace relevante saber la electronegatividad de los elementos dentro de la aleación, siendo esta un indicativo de la diferencia energética entre las capas de los elementos involucrado. Para esto tomamos 2 features:

    12) Electronegatividad promedio

    13) rango de electronegatividad de los elementos

- Estos datos fueron extraídos usando la librería matminer y pymatgen de python

Features de enlace y aproximación magnética

- Para considerar mas a fondo el tipo de enlace que se daba en la aleación y su posible configuración magnética se usaron 6 diferentes features para caracterizar las capas electronicas de los respectivos elementos, estos fueron:

  14) No. Elec. Vale. promedio

  15) Rango de No. De Elec. Val

  16) No. Elec. Vale. Promedio de la capa s

  17) No. Elec. Vale. Promedio de la capa p

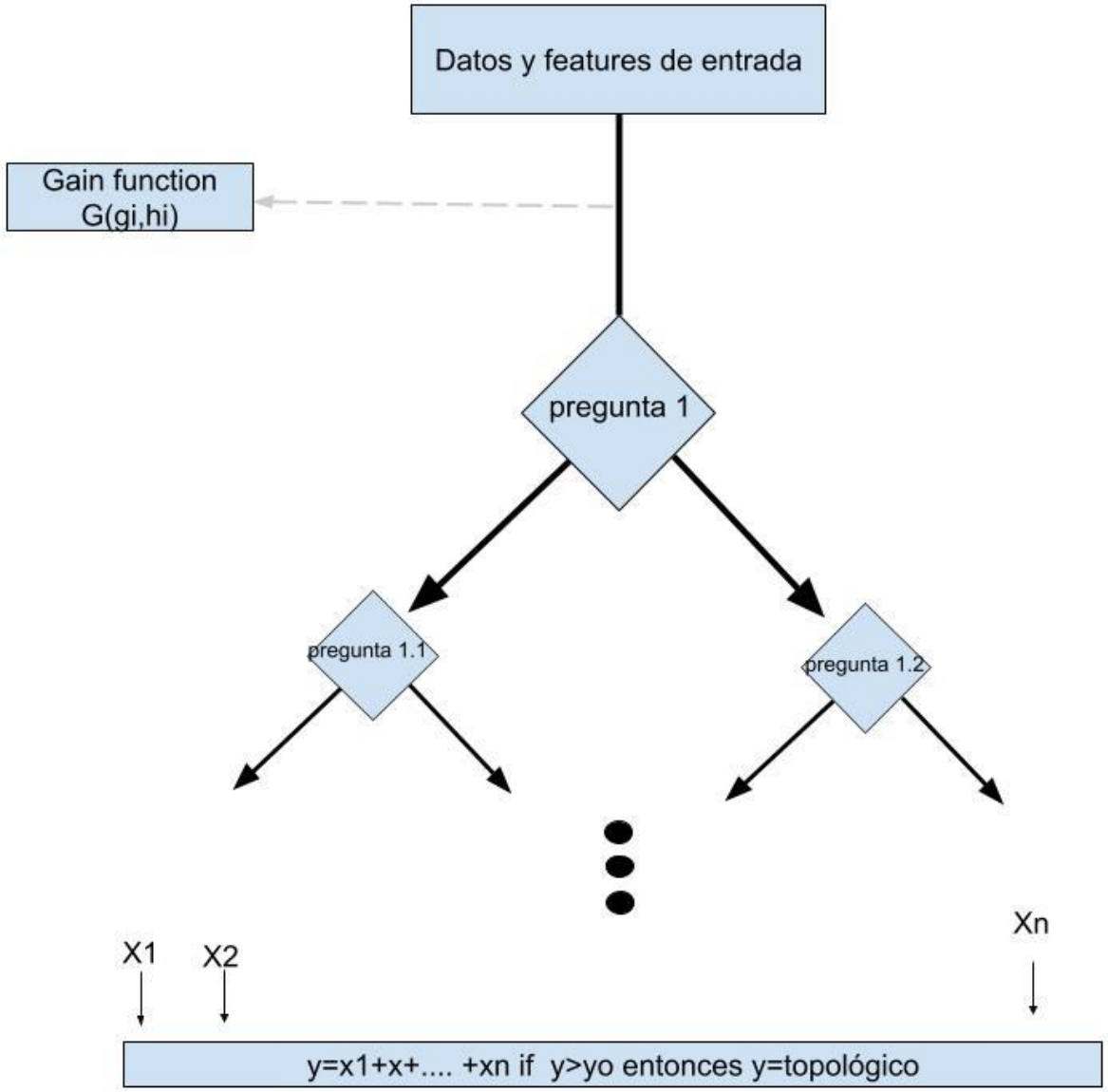
  18) No. Elec. Vale. Promedio de la capa d

  19) No. Elec. Vale. Promedio de la capa f

- Estos datos fueron extraídos usando la librería matminer y pymatgen de python

Construcción de una red de tipo random forest

- Para iniciar la implementación del aprendizaje autónomo a estos sistemas se planteo en 4 formas, la primera de esras como un árbol de decisión.
- Una red de tipo árbol de decisión es un modelo especializado en aprendizaje de problemas binarios (0 o 1), este se basa en general diferentes cadenas de condicionales a las cuales se les asigna un valor relacionado a la condición buscada los cuales se promedian al final y se le asigna una valoración de si o no en forma de regresión logistica


Datos y features de entrada
Gain function
G(gi,hi)
pregunta 1
pregunta 1.1
pregunta 1.2
X1
X2
Xn
y=x1+x+.... +xn if y>yo entonces y=topológico

Código random forest

```
features_topo=["Cubic", "Tetragonal", "Orthorhombic", "Trigonal", "Hexagonal", "Monoclinic","Triclinic",  "Desconocido",
"inver_grup", "magne", "Z_prome", "Z_dife", "Z_sum", "Z_max^4",                    "R_Emp_prome(Angs)",  "R_Emp_dife(Angs)",
"Electronega_prome", "Electronega_dife", "N_val_prome", "N_val_dife", "No_Val_s_prome", "No_Val_p_prome",
"No_Val_d_prome", "No_Val_f_prome", "frac_magne"]

X = df[features_topo].values
y = df["Topologico"].values

rf = RandomForestClassifier(
    n_estimators=500,
    random_state=42,
    class_weight="balanced"
)
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
scores = cross_val_score(rf, X, y, cv=cv, scoring="roc_auc")
```

Resultados para clasificación de topológicos

- AUC promedio: 0.9332313072576925
- Desviación: 0.005948299025415793

Resultados para clasificación de topológicos magnéticos

- AUC promedio: 0.7606666666666667
- Desviación: 0.05964545453690147

Construcción de una red de tipo XGBoost

- Una red de tipo Xboost parte de la base de un árbol de decisión la cual se mejora en cada iteración, modificando en cada paso los valores de la función de ganancia, por medio de un proceso de minimización de una función de error, la cual depende de los valores predichos por el árbol inicial, dando como resultado final la suma de los valores dados por todos las iteraciones

Código XGBoost

```
xgb_model = xgb.XGBClassifier(
    n_estimators=500, #numero de arboles
    max_depth=6, #numero de ramas por árbol (if)
    learning_rate=0.05, #rata de aprendizaje (cuánto influye cada árbol nuevo)
    subsample=0.8, #cantidad de datos que usa el modelo
    colsample_bytree=0.8, #cantidad de features que usa el modelo
    eval_metric="logloss", #métrica de minimización Log Loss.
    random_state=42
)
```

Resultados para clasificación de topológicos

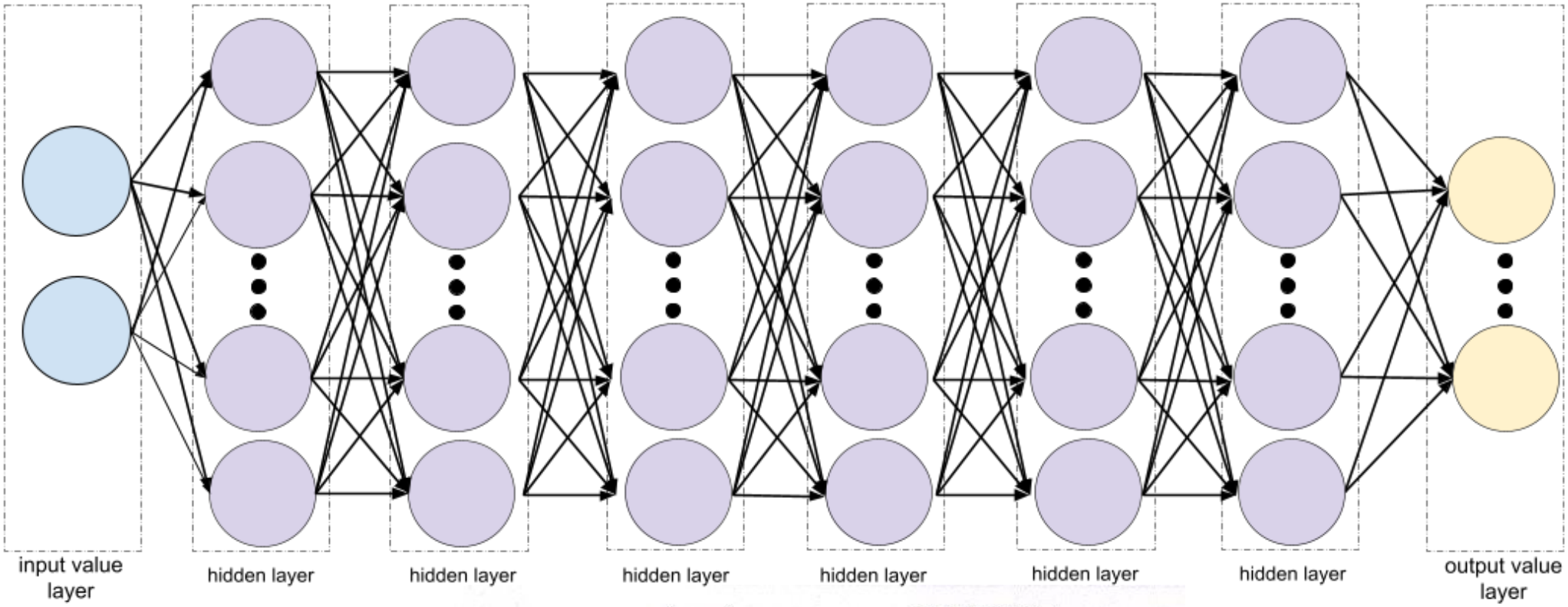
- AUC promedio: 0.9400969498492842
- Desviación: 0.005383799470500899

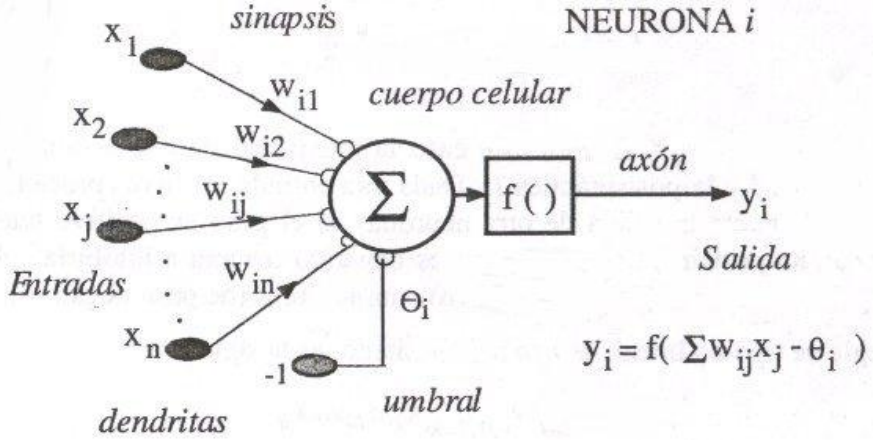
Resultados para clasificación de topológicos magnéticos

- AUC promedio: 0.7788888888888889
- Desviación: 0.08510414825014533

Construcción de una red neuronal

- Definimos una red neuronal como una cadena de operaciones que se le realizan a un paquete de datos los cuales se asocian a pesos en cada operación donde estos pesos se minimizan a razón de una función de perdida


input value layer
hidden layer
hidden layer
hidden layer
hidden layer
hidden layer
hidden layer
output value layer


sinapsis
NEURONA i
cuerpo celular
axón
Salida
Entradas
dendritas
umbral
x_1
x_2
x_j
x_n
w_{i1}
w_{i2}
w_{ij}
w_{in}
Σ
f ( )
y_i
Θ_i
-1
$y_i = f(\Sigma w_{ij} x_j - \theta_i)$

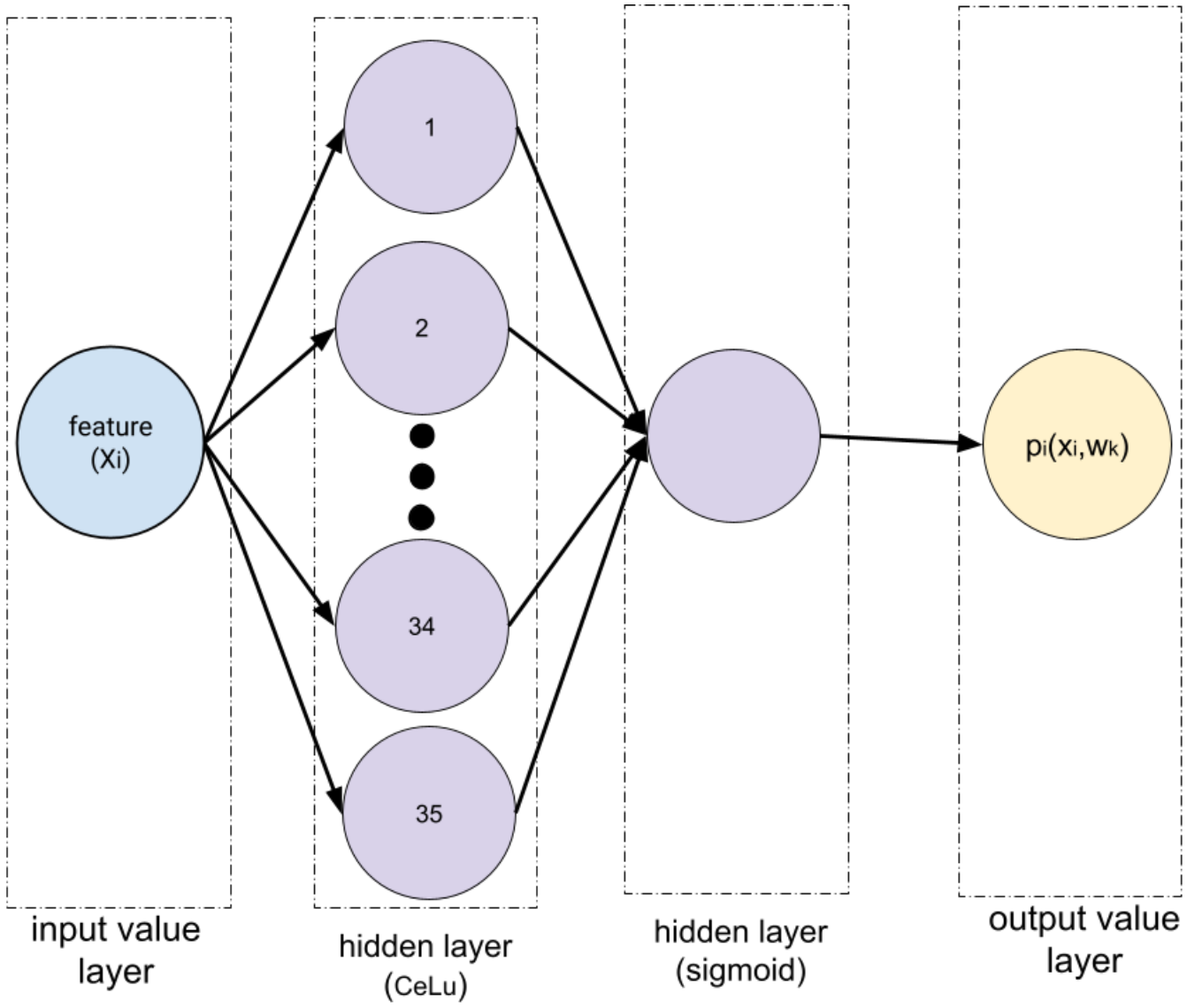
Red de features simples

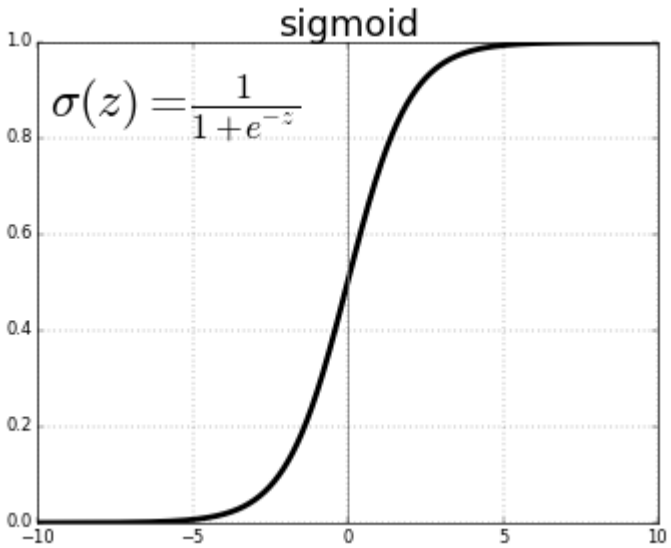
```
tf.keras.backend.clear_session()
 model = keras.Sequential()
 model.add(keras.layers.Dense(units = 35, activation = 'celu',  input_shape=[25]   ))
 model.add(keras.layers.Dense(units = 1, activation = 'sigmoid'))

 model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=[tf.keras.metrics.AUC(name='auc')] # Calculamos el AUC en cada paso
 )
 history = model.fit(
    Xc_train,
    y_train,
    epochs=200,
    batch_size=32,
    validation_data=(Xc_test, y_test),
    verbose=1,
    callbacks=[
      tf.keras.callbacks.EarlyStopping(
         monitor="val_loss",
         mode="min",
         patience=paciencia,
         restore_best_weights=True   ) ]        )
```

Construcción de la red

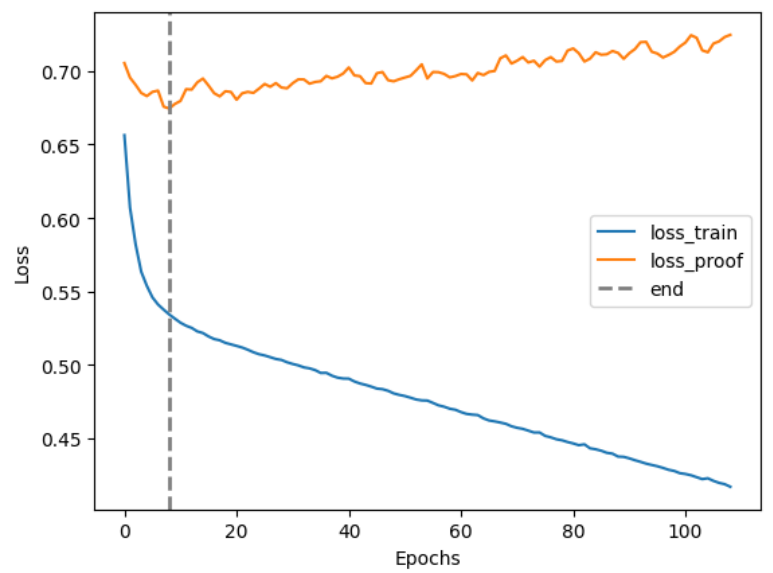
Compilación y minimización

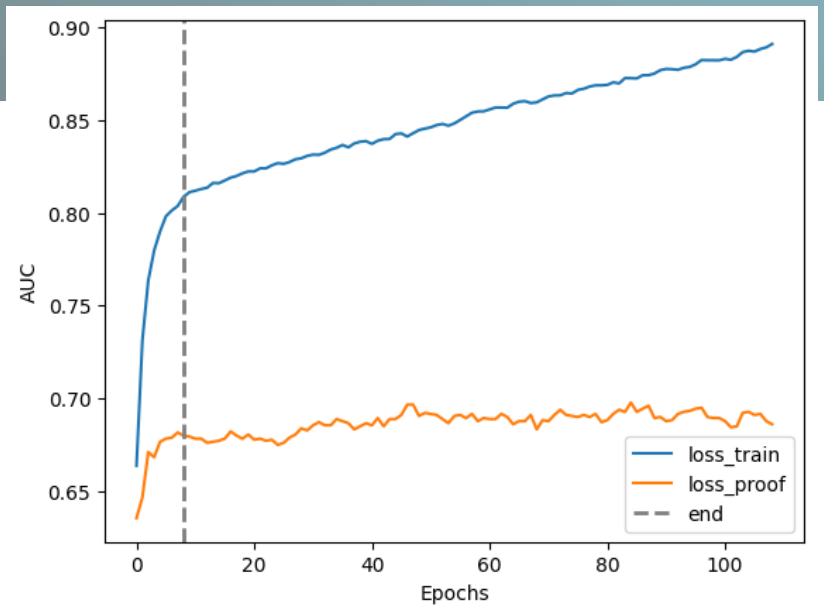
Entrenamiento

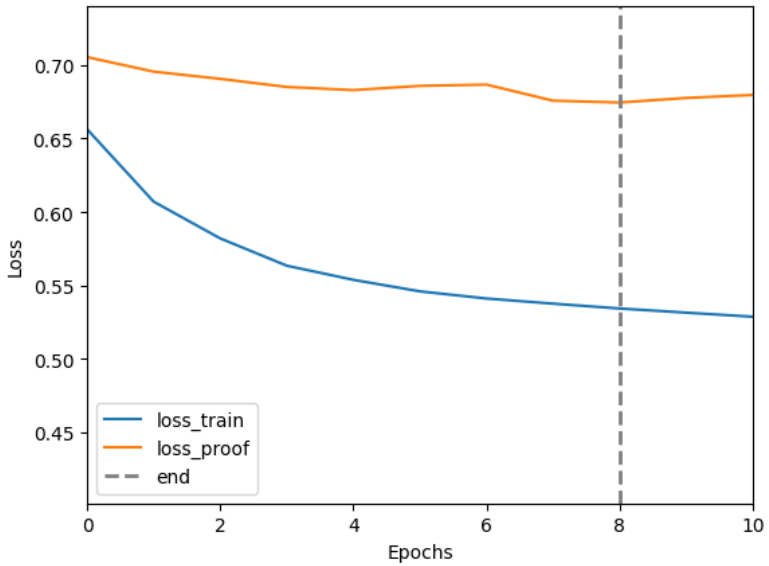


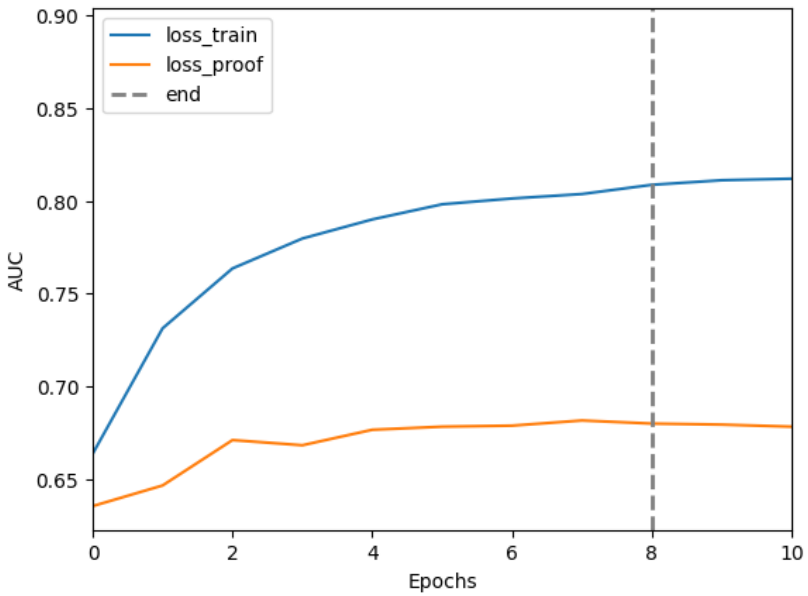


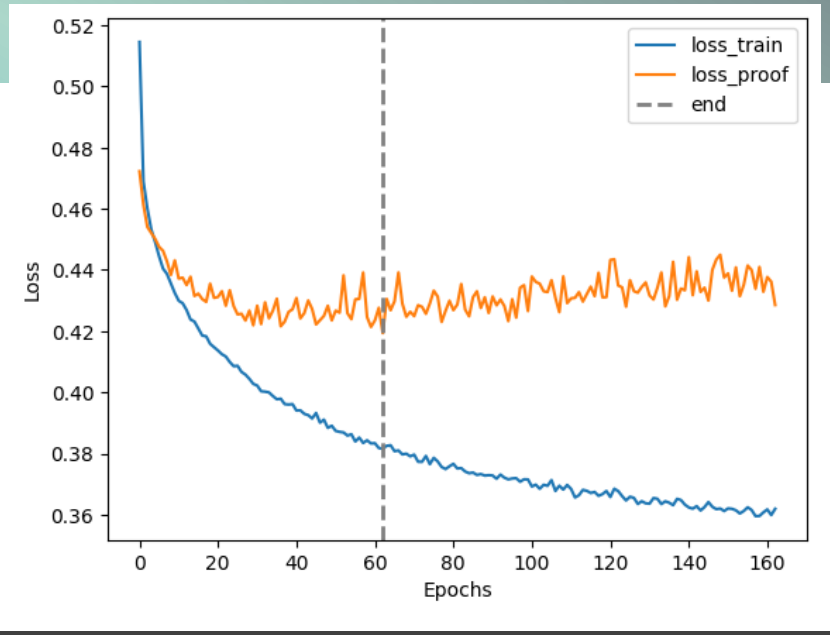
$$L = -\frac{1}{N}\sum_{i=1}^{N} y_i \ln(p_i) + (y_i - 1)\ln(p_i - 1)$$

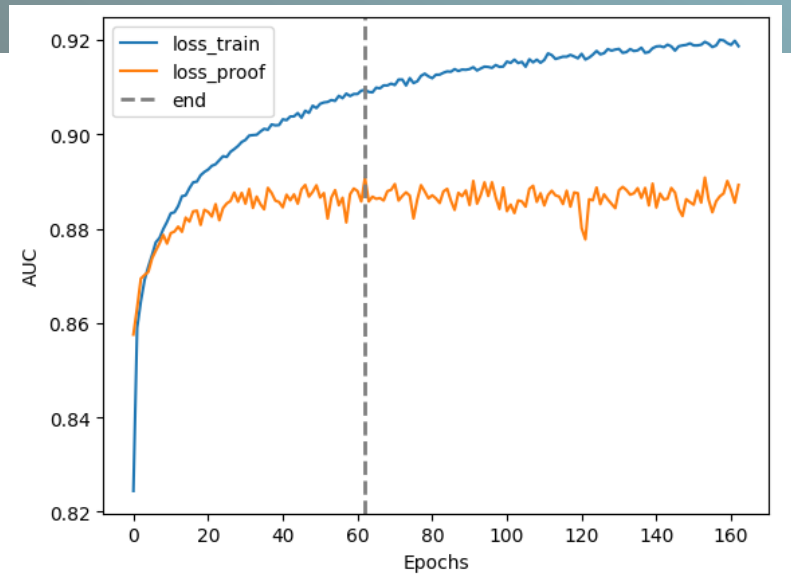

0.70
0.65
0.60
0.55
0.50
0.45
Loss
0
20
40
60
80
100
Epochs
loss_train
loss_proof
end

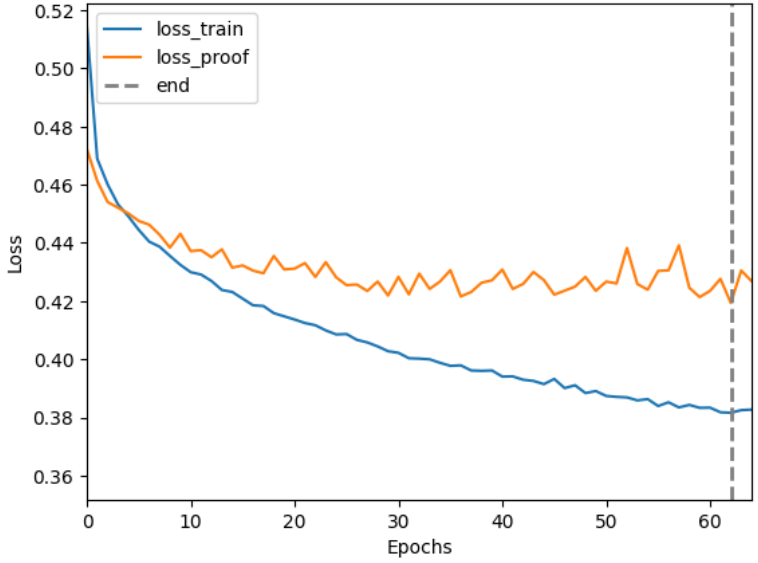

0.90
0.85
0.80
0.75
0.70
0.65
AUC
0
20
40
60
80
100
Epochs
loss_train
loss_proof
end

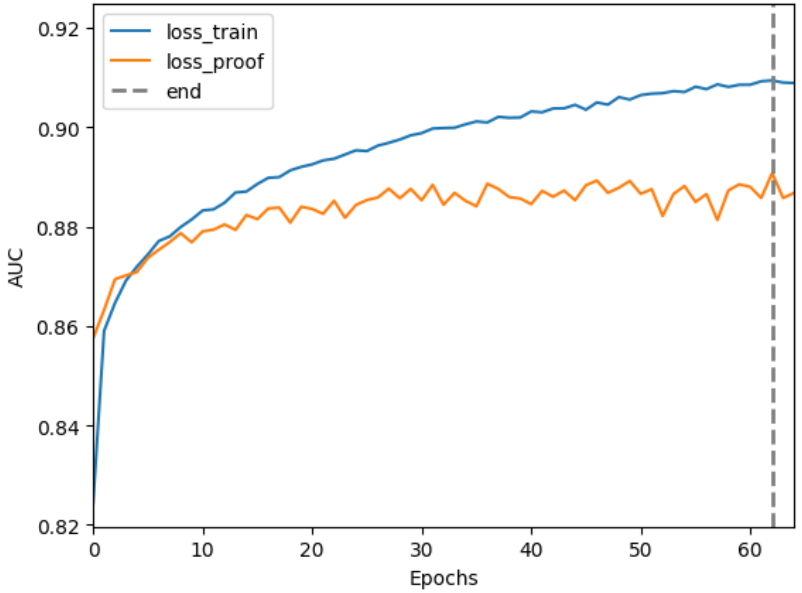

0.70
0.65
0.60
0.55
0.50
0.45
Loss
0
2
4
6
8
10
Epochs
loss_train
loss_proof
end


0.90
0.85
0.80
0.75
0.70
0.65
AUC
0
2
4
6
8
10
Epochs
loss_train
loss_proof
end


loss_train
loss_proof
end
Loss
Epochs


loss_train
loss_proof
end
AUC
Epochs


loss_train
loss_proof
end
Loss
Epochs


loss_train
loss_proof
end
AUC
Epochs

Resultados para clasificación de topológicos

- AUC promedio: 0.8905278220881232
- Desviación: 0.0016188827541855663

Resultados para clasificación de topológicos magnéticos

- AUC promedio: 0.6866666666666666
- Desviación: 0.02718932252299159

Red de features especializados

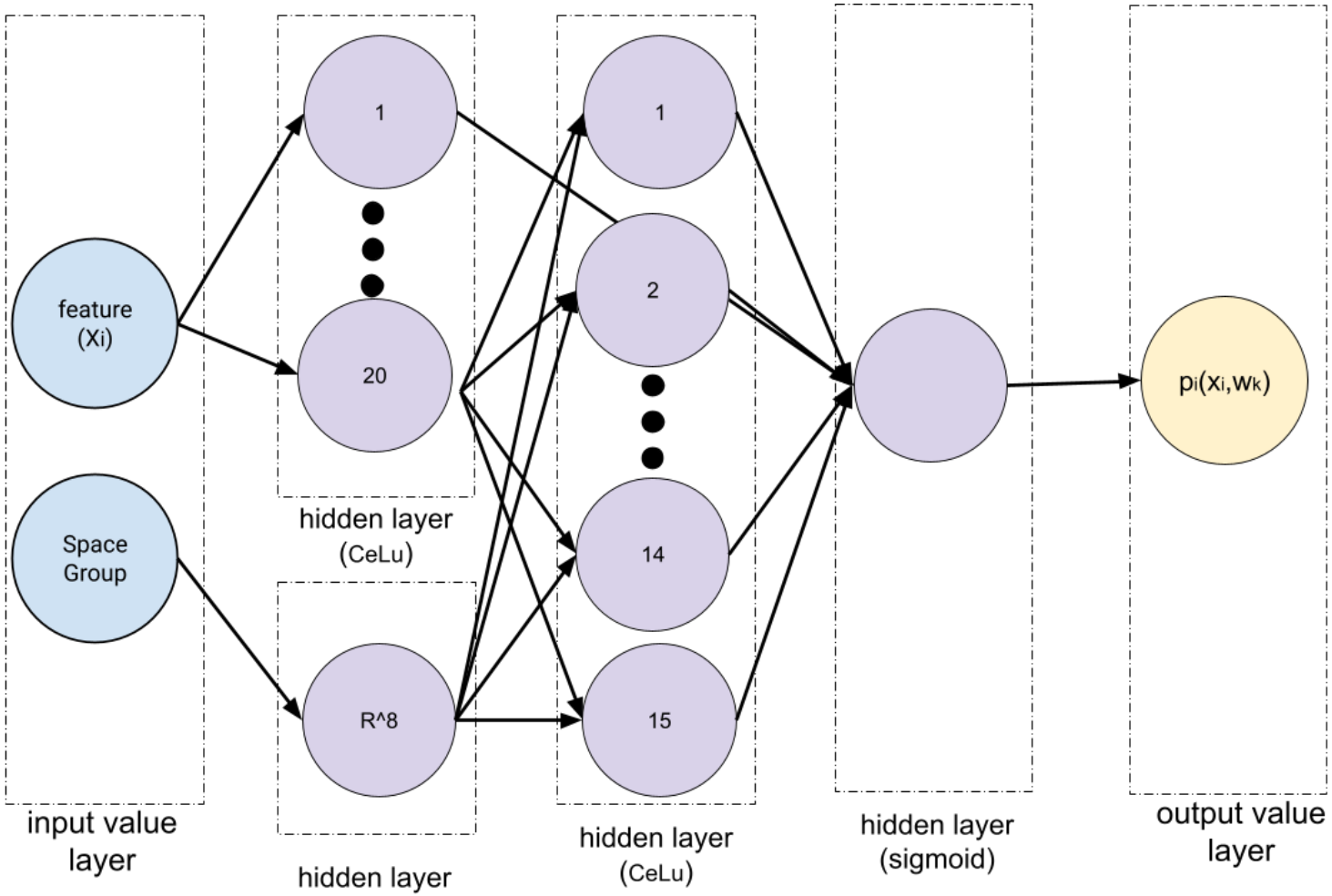
```
+     tf.keras.backend.clear_session()
+     input_cont = Input(shape=(Xc_train.shape[1],), name="cont_features")
+     x_cont = Dense(20, activation="celu",kernel_regularizer=tf.keras.regularizers.l2(1e-4))(input_cont)

+     input_group = Input(shape=(1,), dtype="int32", name="space_group")
+     x_group = Embedding(input_dim=231, output_dim=8, name="group_embedding")(input_group)
+    x_group = Flatten()(x_group)

+     x = Concatenate()([x_cont, x_group])
+     x = Dense(15, activation="celu")(x)
+
      output = Dense(1, activation="sigmoid")(x)
```

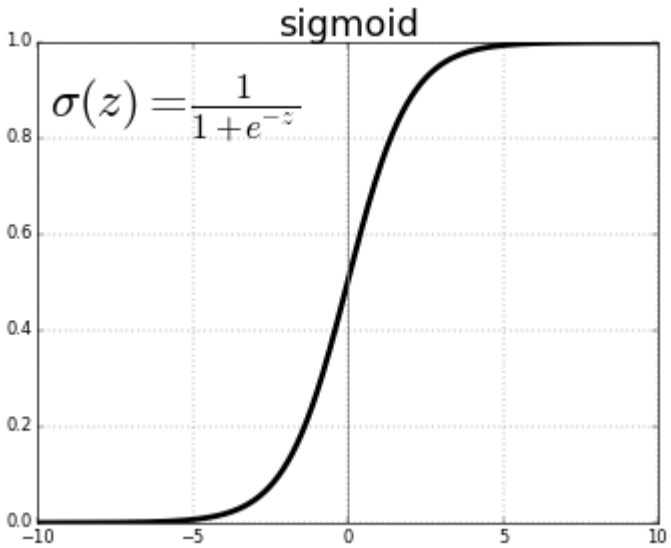
Features físico-químicos

Generamos un vector para el SG

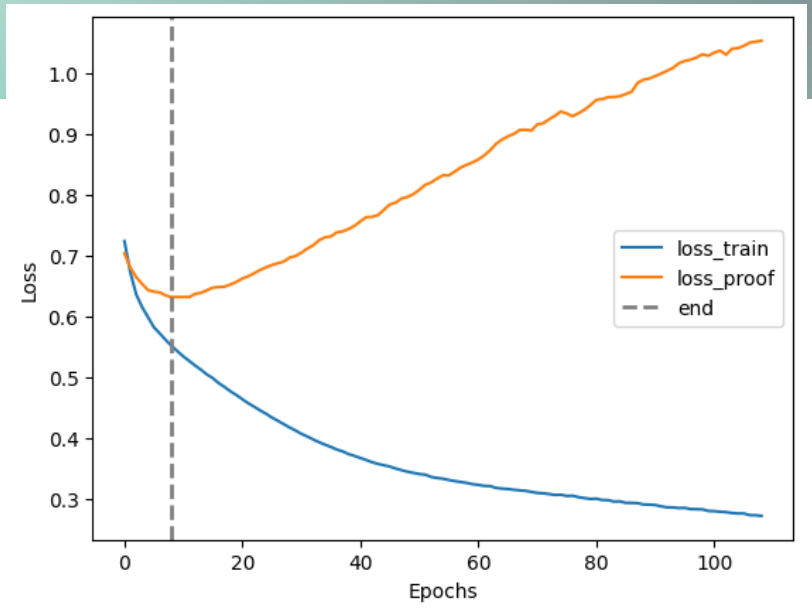
Unimos los features

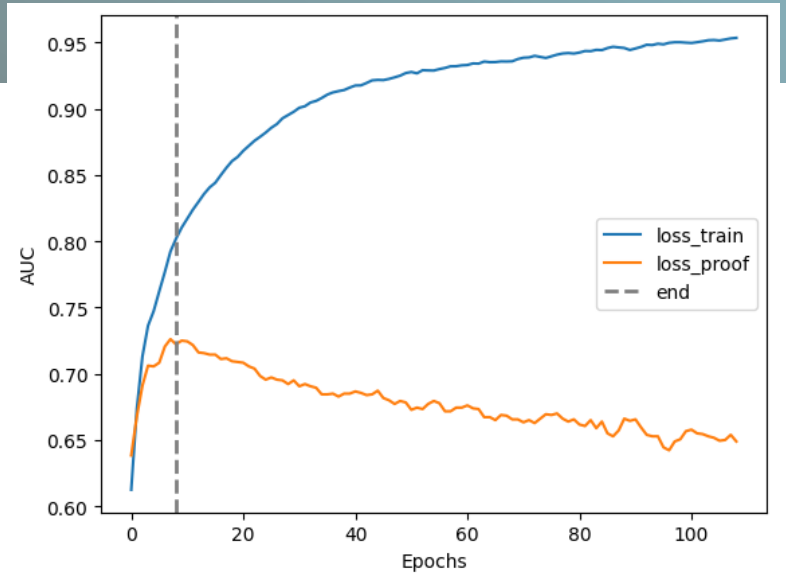
Colapsamos entre 0 y 1

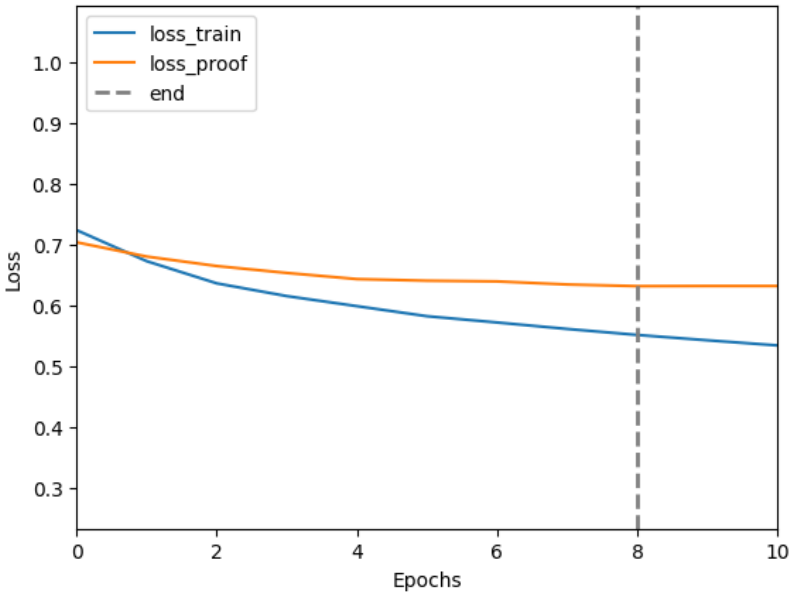


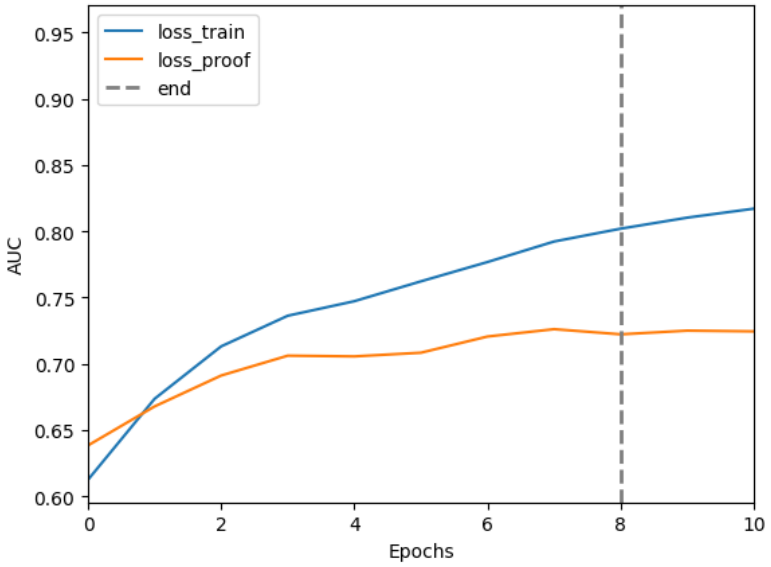


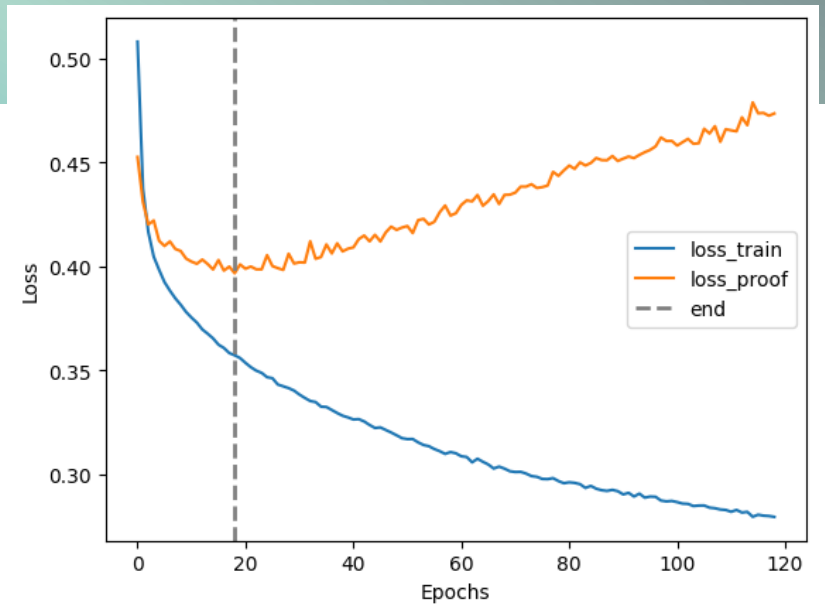
$$L = -\frac{1}{N}\sum_{i=1}^{N} y_i \ln(p_i) + (y_i - 1)\ln(p_i - 1)$$

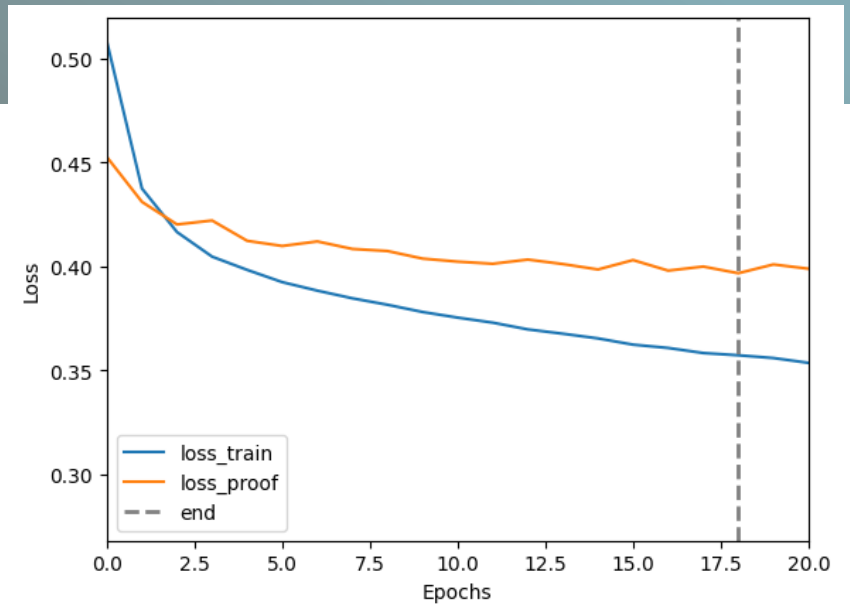

Loss
Epochs
loss_train
loss_proof
end

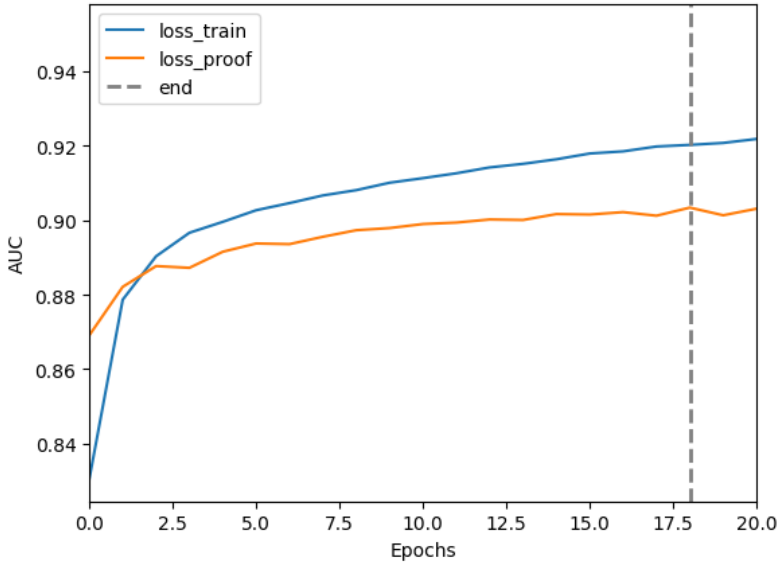

AUC
Epochs
loss_train
loss_proof
end

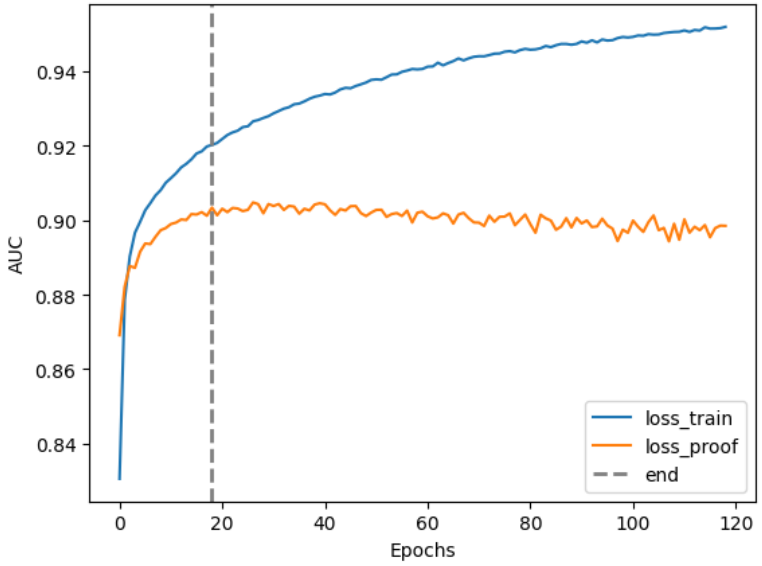

Loss
Epochs
loss_train
loss_proof
end


AUC
Epochs
loss_train
loss_proof
end


0.50
0.45
0.40
0.35
0.30
Loss
0
20
40
60
80
100
120
Epochs
loss_train
loss_proof
end


0.50
0.45
0.40
0.35
0.30
Loss
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epochs
loss_train
loss_proof
end


0.94
0.92
0.90
0.88
0.86
0.84
AUC
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epochs
loss_train
loss_proof
end


0.94
0.92
0.90
0.88
0.86
0.84
AUC
0
20
40
60
80
100
120
Epochs
loss_train
loss_proof
end

Resultados para clasificación de topológicos

- AUC promedio: 0.9400969498492842
- Desviación: 0.005383799470500899

Resultados para clasificación de topológicos magnéticos

- AUC promedio: 0.6991110920906067
- Desviación: 0.015478384019948931

FIN, GRACIAS POR LA ATENCIÓN :)))